

Naming Conventions

Shop360Bike Medallion Data Warehouse

Microsoft SQL Server | Olamide Akanni

This document explains how we name schemas, tables, views, columns, procedures, and script files across the warehouse. It covers the conventions we actually use and notes three places where we've made deliberate exceptions.

General Principles

- Case: snake_case throughout. Lowercase with underscores between words.
- Language: English for all object names.
- Reserved words: never used as object names.
- Abbreviations: kept out of table and column names, but used in procedure names (layer prefixes shortened to brz and slv).
- Plurality: dimensions and facts use the plural form of the business entity, like dim_customers.

Schema Naming

Four schemas, one for each medallion layer plus an orchestration layer:

Schema	Purpose	Contains
bronze	Raw data exactly as received from the source	Tables and load procedures
silver	Cleansed and standardized data	Tables and load procedures
gold	Business-ready star schema	Views and one generated table
dbo	Pipeline orchestration and logging	Tables and procedures

We use dbo for orchestration rather than a separate schema because logging needs to touch all layers. It sits outside the medallion pattern and is used by all of them.

Table Naming Conventions

Bronze Layer

Bronze tables preserve the original source system name followed by the table name, unchanged. This keeps the answer to "what did the source send us?" always visible in the schema.

Pattern: <source_system>_<entity>

Table	Source	Original Name
crm_cust_info	CRM (CSV)	cust_info.csv
crm_prd_info	CRM (CSV)	prd_info.csv
crm_sales_details	CRM (CSV)	sales_details.csv
erp_cust_az12	ERP (CSV)	CUST_AZ12.csv
erp_loc_a101	ERP (CSV)	LOC_A101.csv
erp_px_cat_g1v2	ERP (CSV)	PX_CAT_G1V2.csv
inventory	BikeShopOLTP	dbo.inventory
fx_rates	exchangerate-api.com	REST response
web_events	Clickstream	Generated events

Exception 1: The last three tables have no source prefix. By strict convention they'd be `oltp_inventory`, `api_fx_rates`, and `web_web_events`. We dropped the prefix because each source contributes exactly one table, making the prefix unnecessary noise, and `web_web_events` just looks wrong.

Silver Layer

Silver mirrors bronze exactly, using the same names. This makes the lineage from bronze to silver obvious and lets you trace a transformation by comparing two tables with the same name in different schemas. Renaming happens only when we reach gold, where source-style names become business-style names.

Gold Layer

Gold names are business-facing and use a prefix describing what the object does in the model.

Pattern: `<category>_<business_entity>`

Prefix	Meaning	Examples
<code>dim_</code>	Dimension	<code>dim_customers</code> , <code>dim_products</code> , <code>dim_currency</code> , <code>dim_date</code>
<code>fact</code>	Fact table	<code>fact_sales</code> , <code>fact_inventory</code> , <code>fact_fx_rates</code>
<code>vw</code>	Helper view	<code>vw_fx_rates_readable</code>
<code>report_</code>	Pre-aggregated report table	(not yet used)

Exception 2: We added the `vw_` prefix as a fourth category. It marks objects in gold that sit outside the star schema, like `vw_fx_rates_readable` which exposes FX rates with currency codes instead of surrogate keys for ad-hoc queries. This keeps the star schema unambiguous.

Note: `dim_date` and `dim_currency` are both singular (not plural). `dim_date` follows universal convention, and `dim_currency` sounds better than `dim_currencies` with its ISO code reference.

Column Naming Conventions

Surrogate Keys

Every dimension primary key uses the `_key` suffix and is generated in gold with `ROW_NUMBER()`. Facts reference the surrogate key, never the source ID, so changes to a source's numbering scheme can't break warehouse relationships.

Pattern: `<entity>_key`

Column	Defined In	Used By
<code>customer_key</code>	<code>dim_customers</code>	<code>fact_sales</code> , <code>fact_web_events</code>
<code>product_key</code>	<code>dim_products</code>	<code>fact_sales</code> , <code>fact_inventory</code>
<code>currency_key</code>	<code>dim_currency</code>	<code>fact_fx_rates</code> (twice)
<code>date_key</code>	<code>dim_date</code>	<code>fact_inventory</code> , <code>fact_fx_rates</code>

When a fact table references the same dimension in multiple roles, the key name includes the role. For example, `fact_fx_rates` has `base_currency_key` and `target_currency_key`, both pointing to `dim_currency`.

Exception 3: `fact_sales` uses actual DATE values (`order_date`, `shipping_date`, `due_date`) instead of integer date keys. It joins `dim_date` on `full_date` rather than `date_key`. This breaks the surrogate key pattern used by the other facts, and we've added creating those keys to the roadmap.

Natural Keys

When a dimension includes the source system identifier, the column keeps its original name with no suffix. `dim_customers` has `customer_key` alongside `customer_id`; `dim_products` has `product_key`, `product_id`, and `product_number`. No suffix means it's a natural key.

Business Columns

- Named for what the business calls them, not the source. `cst_firstname` in silver becomes `first_name` in gold.
- No source system abbreviations in gold. Prefixes like `cst_`, `prd_`, and `sls_` are dropped at the gold boundary.
- Boolean columns get the `is_` prefix: `is_weekend`.
- Dates use the `_date` suffix (`full_date` is the exception, named for its role as the complete date alongside `date_key`).

Technical Columns

System-generated metadata uses the `dwh_` prefix to keep technical columns separate from business ones.

Column	Type	Purpose
<code>dwh_create_date</code>	DATETIME2	Timestamp when the row was loaded into silver. On all silver tables.

Stored Procedure Naming

Layer Batch Loaders

Pattern: `load_<layer>`

Loads every table in a layer that comes in the CRM and ERP batch.

Procedure	Loads
<code>load_bronze</code>	All six CRM and ERP tables from CSV
<code>load_silver</code>	All six CRM and ERP tables from bronze

Per-Source Loaders

Pattern: `load_<layer_abbrev>_<entity>`

Loads a single source arriving outside the CSV batch. We abbreviate the layer to `brz` or `slv` to keep names readable.

Procedure	Loads
<code>load_brz_inventory</code>	Stock snapshots from BikeShopOLTP
<code>load_slv_inventory</code>	Stock snapshots from bronze
<code>load_slv_fx_rates</code>	Exchange rates from bronze
<code>load_slv_web_events</code>	Clickstream events from bronze

We use abbreviations because `load_bronze_inventory` reads awkwardly next to `load_bronze`, and dropping the layer entirely gives two procedures called `load_inventory` in different schemas. Abbreviating is the cleanest option, but it means these two patterns aren't interchangeable.

Orchestration Procedures

Orchestration procedures are verb phrases describing what they do. They don't load anything themselves, so `load_` would be misleading.

Procedure	Does
<code>log_and_run</code>	Executes a procedure, times it, and logs the result
<code>run_full_pipeline</code>	Runs all load procedures in order with one run ID

Script File Naming

SQL Files

SQL files use a prefix describing what they create:

Prefix	Contains	Examples
ddl_	CREATE TABLE/VIEW	ddl_bronze.sql, ddl_silver.sql, ddl_gold_views.sql
proc_	CREATE PROCEDURE	proc_load_brz_crm_erp.sql, proc_run_full_pipeline.sql
quality_checks_	Validation queries	quality_checks_silver.sql, quality_checks_gold.sql

Python Scripts

Python files use verb_noun naming, describing what they do:

Script	Action
fetch_fx_rates.py	Gets daily rates from the API into bronze
generate_web_events.py	Creates synthetic clickstream events
send_slack_alert.py	Posts pipeline results to Slack

Note: The external source DDL files (ddl_b_ext_source.sql and ddl_s_ext_source.sql) abbreviate the layer to a single letter. They could be renamed to ddl_bronze_ext_source.sql and ddl_silver_ext_source.sql to match procedure naming, but we've left them as a known inconsistency.